

An HTTP Server and RESTful API in Python

Fougeray Paul

Department of Computer Science
UiT The Arctic University of Norway
Tromsø, Norway
pafou7668@uit.no

Abstract—This report presents an HTTP/1.1 server written in Python. I use the low-level `socketserver` module instead of a ready-made HTTP library. The server exposes a static file endpoint with `GET` and `POST`. It also exposes a RESTful `/messages` endpoint with `GET`, `POST`, `PUT` and `DELETE` on JSON resources. Messages persist to disk between restarts. The assignment leaves many edge cases open, so I document and justify my choices for them in this report. I verify the solution with an automated test suite and with Wireshark packet captures.

Index Terms—HTTP, REST, socket programming, JSON, persistence.

I. INTRODUCTION

The goal of this assignment is to understand HTTP by implementing it on raw TCP sockets, instead of using a ready-made HTTP library. I then build a small REST API on top of the same server. Working at this level makes several hidden details explicit. I need to know where a request ends on the wire. I need to know the body length in advance. I need to build a response that a client can parse without ambiguity.

I use one generic request handler for both parts of the assignment: the static file endpoint and the `/messages` REST endpoint. This follows the assignment’s advice to write a single server. Both parts share the same parsing and response code. Only the routing step differs between them.

Much of this report explains and justifies my choices for the edge cases the assignment leaves open. For example: what should happen on a `PUT` request with no `id`? What should happen on a `POST` to a message URL that already names a specific `id`?

A. Outline

The rest of this report is organized as follows. Section II covers the background needed for the rest of the report. Section III gives a high-level view of my design. Section IV explains my implementation in detail. Section V describes my test methodology and results. Section VI discusses the design choices behind my handling of edge cases. Section VII concludes the report and lists possible future work.

II. THEORETICAL BACKGROUND

A. HTTP message structure

An HTTP/1.1 message has a start-line, a set of header fields, a blank line, and an optional body [2]. A connection can stay open for several requests, so a server cannot read the body “until the socket closes”: it needs to know the body length

in advance, before it can safely stop reading and move on to the next request. The `Content-Length` header gives this length in bytes [1], regardless of how many other headers are present or in what order.

B. Status codes used

My server returns six status codes. 200 OK means the request succeeded. 201 Created means I created a new message. 400 Bad Request means the request or its JSON body is invalid. 403 Forbidden means the resource exists but I refuse to expose it. 404 Not Found means the resource does not exist. 501 Not Implemented means I do not support that HTTP method. I use 501 rather than 400 for an unsupported method such as `HEAD`: the request line is still valid HTTP, my server simply has no behaviour for that verb. I use 400 only when the request is malformed at the protocol level, for example a request line without three fields.

C. Resources vs. files

REST treats a resource as separate from a file on disk. `/messages` is a resource backed by a JSON document, not a path a client could browse to on the filesystem the way `/index.html` is. This is why I handle the static file endpoint and `/messages` with separate application logic, even though both go through the same request-parsing code.

D. Persistence

Persistence means more than writing to a file: a crash during a write must not corrupt the file that is already on disk. The standard way to avoid this is to write the new state to a temporary file first, then atomically replace the real file with it. At any point in time, the file on disk is then either the old, complete version, or the new, complete version, never something in between. Section IV-E describes how I apply this to `messages.json`.

III. DESIGN

A. Overview

I use a single request-handler class for the whole server, following the assignment’s advice to write one server rather than duplicate code between the static file part and the REST part. Every request goes through the same steps: read the request line, read the headers, read exactly `Content-Length` bytes of body, look up a handler for the method and path, then send back a response with a `Content-Length` header (and

a Content-Type header whenever the body is non-empty). Only the routing step differs between the static file part and /messages.

B. Static resource routing

I design the static file part around a whitelist: only a small, fixed set of routes (`/`, `/index.html`, `/favicon.ico`, `/test.txt`) can be served or written to. Anything else is refused by default. A whitelist cannot accidentally expose a new file added to the project directory later, while a blacklist has to be updated by hand every time. GET and POST use different route sets on purpose: any whitelisted file can be read, but only `/test.txt` can be written to.

C. The /messages resource

I model a message as a JSON object with two fields, `id` and `text`. The server always assigns the `id`; I never trust one from the client, since it is what identifies a message across requests, and letting a client choose it would risk collisions with other messages. I address a message in two ways: through the request body (an `id` field, as the base assignment requires) on the `/messages` collection, or through the URL itself, `/messages/<id>`, closer to how most real REST APIs work. I treat persistence as a design decision, not an afterthought: since state must survive a server restart, every change to the message list goes through an explicit load-then-save step, rather than staying only in memory.

IV. IMPLEMENTATION

A. Technical Details

I write the server in Python 3, using only the standard library (`socketserver`, `json`, `urllib.parse`, `tempfile`, `email.utils`). I do not use an HTTP framework, as the assignment requires. `server.py` has three layers: the `MyTCPHandler` class parses requests and writes responses; a small decorator-based router, the `API` class, maps a method and path to a handler function; and one handler function per route holds the actual application logic.

I keep the router simple: two Python dictionaries and a few decorator methods (`@api.get`, `@api.post`, `@api.get_id`, and so on), not a general regular-expression engine. This is enough to express every route in this assignment: a few fixed static paths, the `/messages` collection, and the one dynamic route, `/messages/<id>`.

B. Request parsing

I read the request line and headers line by line with `rfile.readline()`. I tolerate one leading blank line, as RFC 9112 section 3 allows, then require a well-formed request line. Any malformed line raises a `_BadRequest` exception, which I catch once in `handle()` and turn into a 400 response instead of crashing the handler. I read the body with exactly `int(Content-Length)` bytes via `rfile.read()`, never by reading until the socket closes, for the reasons in Section II-A. I detect a method outside GET/POST/PUT/DELETE before any routing and answer with 501.

C. Static resource routing and path traversal protection

Each whitelisted route (`/`, `/index.html`, `/favicon.ico`, `/test.txt`) resolves to one fixed filename inside `BASE_DIR`, so a request for one of them can never escape that directory. Any GET request that matches none of these exact routes falls through to a generic fallback. I resolve the target path with `os.path.realpath()` and check it against `BASE_DIR` before touching the filesystem; anything outside `BASE_DIR` gets 403, before I even check whether the file exists. A file that exists but is not whitelisted, such as `server.py`, also gets 403 rather than 404. This way, a client cannot tell a forbidden file from a missing one.

D. The /messages REST API

GET `/messages` returns the full list as a JSON array. POST `/messages` checks that the body is a JSON object with a non-empty `text` field, assigns the next `id`, and returns 201 with the new object. PUT/DELETE on `/messages` take the `id` from the JSON body, as the base assignment requires. The `id-in-URL` variants, GET/PUT/DELETE on `/messages/<id>`, are separate routes registered with `@api.get_id/@api.put_id/@api.delete_id`, and reuse the same `find_message()` and `save_messages()` helpers. I also register POST `/messages/<id>`, only to answer 403: creating a message only makes sense on the collection endpoint. Every change goes through `save_messages()`, which writes the full state to `messages.json`.

E. Persistence implementation

`save_messages()` writes the new state to a temporary file with `tempfile.mkstemp()`, then calls `os.replace()` to atomically swap it in for `messages.json`. If anything fails, I remove the temporary file and re-raise the exception, leaving the previous `messages.json` untouched. `load_messages()` falls back to an empty list, with `next_id` at 1, if `messages.json` is missing or corrupt, so a first run, or a manually deleted file, does not crash the server.

F. Extra credits implemented

I add `Server` and `Date` headers to every response, in `_send()`. I use a whitelist, not a blacklist, for path traversal protection, as Section IV-C describes. I implement `/messages/<id>` addressing for GET, PUT and DELETE. I leave one thing out: a real 3xx redirect from `/` to `/index.html`. The base requirement only asks for the body of `index.html` on GET `/`, which I already satisfy without a redirect, so I treat this as a conscious trade-off, not an oversight.

V. EXPERIMENTS AND RESULTS

A. Automated testing

The provided `test_client.py` drives the handler through a second, in-process server. It checks 12 behaviours: a valid response code, the body and headers of GET `/`, the 404

and 403 status codes, protection against directory traversal, and the behaviour of POST to `/test.txt`. It runs the suite in sequential order, then in random order; the random pass only runs if the sequential pass fully succeeds. I applied the errata fix for this file before running it: every request target now starts with a slash. All 12 tests passed, in both sequential and random order (12/12, 100%).

B. Automated testing of /messages

`test_client.py` does not test `/messages`, so I wrote a second suite, `test_messages.py`, in the same style: boolean- returning test functions with a docstring, run sequentially then in random order. It covers GET/POST/PUT/DELETE on the collection and on `/messages/<id>`, and every edge case from Section VI: a missing text or id field, a non-existent id, malformed JSON, an empty body, a client-supplied id being ignored, POST to an id URL being forbidden, and a created message being readable back through `load_messages()`. Each test creates and removes its own message, which keeps the suite safe to run in any order. All 23 tests passed, in both sequential and random order (23/23, 100%). This suite runs its own server on a separate port, `localhost:54322`, so I can run it independently of `test_client.py`, on 54321.

C. Packet captures

I use Wireshark to check the network traffic directly. This confirms that the header and body invariants from Section IV hold on the wire, not only in the client library’s already-parsed view. I capture both examples on the loopback interface (`lo`), filtered on `tcp.port == 8080`, and display the underlying TCP stream with Wireshark’s “Follow TCP Stream” view.

Figure 1 shows a GET `/` request and its response on the static file endpoint. The TCP handshake appears above the stream view, followed by the GET `/` HTTP/1.1 request line and a 200 OK response. This response carries Content-Length, Content-Type, Server and Date headers, as required. Figure 2 shows a POST `/messages` request and its response on the REST endpoint: the JSON body sent by the client, `{"text": "Example text"}`, and the 201 Created response, which includes the server-assigned id.

VI. DISCUSSION

A. Unsupported and malformed methods

A method outside GET/POST/PUT/DELETE must not crash the server. This includes HEAD and a made-up verb like PIZZA. I answer these with 501 Not Implemented, rather than ignoring them or answering with 400: the request line is still valid HTTP, my server simply has no behaviour for that method, which is exactly what 501 means. A structurally invalid request is different, for example a request line without three fields, or a header line without a colon. I answer these with 400 Bad Request, since I cannot parse them as HTTP at all.

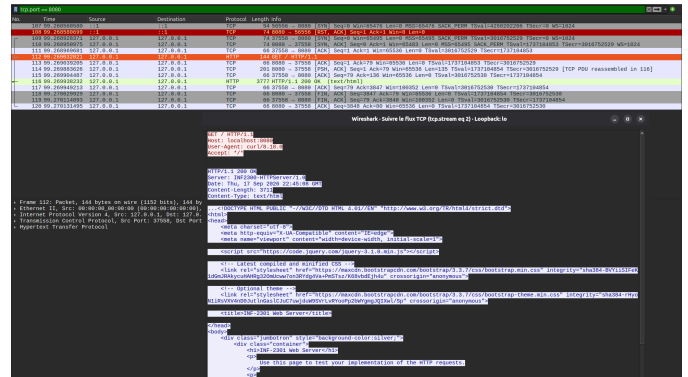


Fig. 1. Wireshark capture of a GET `/` request/response on the static file endpoint (loopback interface, `tcp.port == 8080`).

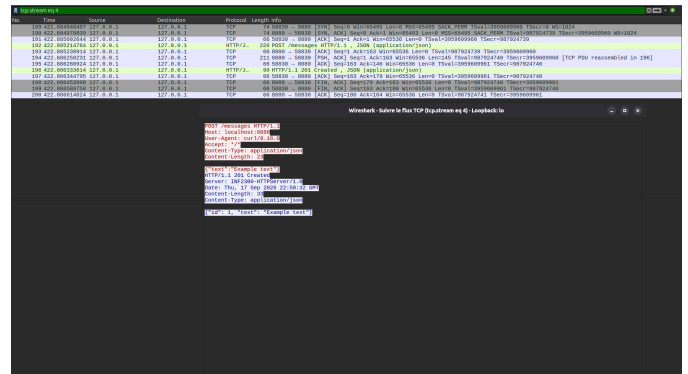


Fig. 2. Wireshark capture of a POST `/messages` request/response on the REST endpoint, showing the JSON payload and the 201 Created response.

B. Static file edge cases

A POST to any file other than `/test.txt`, such as `server.py`, gets 403: writability is opt-in, like readability, and only `/test.txt` is on the writable whitelist. A GET to `/test.txt` before it exists gets 404, since the file genuinely does not exist yet. This differs from GET `/messages`, which returns 200 with an empty array: the `/messages` resource exists, it just has no content yet, while `/test.txt` as a file does not exist until the first successful POST. A directory traversal attempt, such as requesting `../README.md`, gets 403, verified both by `test_client.py` and manually with `curl`. I prefer a whitelist with a realpath containment check over blacklisting patterns like `".."`: a blacklist has to anticipate every way to manipulate a path, such as URL encoding or mixed separators, while checking that the real path stays inside `BASE_DIR` handles all of them at once.

C. /messages edge cases

I ignore any `id` field a client sends in a POST body: the server always assigns the id itself, since trusting a client-supplied id could let one client overwrite a message it does not own. A POST with no `text` field, or a PUT with an id but no `text` field, gets 400: these are malformed requests, not missing resources. A PUT with no `id` at all also gets 400, not 404, since the request itself is invalid, independent of any id;

this differs from a PUT/DELETE with a valid id that simply does not exist, which gets 404. An empty body, or a body that is not valid JSON, also gets 400. Finally, a POST to /messages/<id> gets 403: creating a message at a URL for a specific, existing id does not make sense, so only the collection endpoint, /messages, accepts new messages.

D. Security considerations

I apply the whitelist-over-blacklist choice from Section VI-B as a general principle: a route or file that is not explicitly listed is forbidden by default, so a new file added to the project directory later is never exposed by accident. The directory traversal test confirms this. I consider authentication and rate limiting out of scope for this assignment.

E. Persistence and crash safety

I write to a temporary file and then rename it into place, instead of writing directly to `messages.json`: a crash during a direct write could leave a truncated, unreadable JSON file, silently discarding every earlier message. The atomic rename removes this risk entirely: at any point, `messages.json` is either the old, complete state, or the new, complete state. One limitation remains: I do not lock `messages.json` against two simultaneous requests. This is acceptable for the assignment's single-client testing model, but I note it here as a known limitation.

VII. CONCLUSION

This report presents an HTTP/1.1 server built on `socketserver`, exposing a static file endpoint and a persistent, RESTful /messages endpoint through one generic request handler. The solution meets the assignment's requirements. I verify it with two automated test suites, 12/12 and 23/23, both passing in sequential and random order, and with Wireshark packet captures. I implement three extra credits: whitelist-based path traversal protection, `Server` and `Date` headers, and /messages/<id> addressing. Two limitations remain for future work: a locking mechanism for `messages.json` under concurrent requests, and a real 3xx redirect from / to /index.html.

REFERENCES

- [1] IETF. *HTTP Semantics*. RFC 9110. 2022. URL: <https://www.rfc-editor.org/rfc/rfc9110>.
- [2] IETF. *HTTP/1.1*. RFC 9112. 2022. URL: <https://www.rfc-editor.org/rfc/rfc9112>.